One rickshaw, 10 styles
the recipes behind every look

The reel is **one** 3D auto-rickshaw built in three.js from about 45 simple parts: extruded outlines, cylinders, a torus for each tyre. No Blender, no 3D files, no stock. Every look is the same frame with a different material and a small 2D pass on top, and a scan-line wipe carries you from one look to the next. Spin it yourself and tap through the styles at **thenispace.com/styles**. Below: what each style is, how it is done, and the one-line brief to hand Claude Code.

THE BASE: A RICKSHAW FROM SIMPLE PARTS

- **Shape list, not a model.** Lower shell = a top-view outline (rectangle that tapers into a round nose) extruded 0.5 high with soft bevels. A half-cylinder mudguard over **one** front wheel, two tyres at the back (each a torus + a hub). A tilted windshield frame (an extruded rectangle with a hole, glass inside), six round pillars, a bevelled yellow canopy with a soft-top back, a bench and seat back, handlebar with grips, a chrome headlight ring with a warm lens, mirrors, a meter, number plates with real text, a bumper and an exhaust.
- **Colours = Delhi auto.** Green body `#1b7f3b`, yellow canopy `#f5b82e`, dark seats, chrome fittings.
- **Light.** One key light with a soft 2048 px shadow map, a cool fill, a rim light from behind and a hemisphere light; ACES tone mapping on the shaded looks. The shadow on the floor is what makes clay, cartoon and voxel feel solid.
- **Camera.** Each look has its own shot: a start angle, a slow turn and a 3.5 % push-in over 2.1 s (4 beats at 112 BPM). Styles 1–6 and 8–10 start from a 3/4-front angle, blueprint is a pure side elevation.
- **Cuts.** Every switch is a scan-wipe: the new look sweeps in left to right behind a bright line in 0.34 s, on the beat.

---

01 WIREFRAME

What: only the edges, cyan on black, over a 3D floor grid.

How: `EdgesGeometry(geometry, 15°)` + `LineSegments` in `#19e6ff`; the faces stay at 9 % opacity so the volume reads; a `GridHelper` floor at 16 %. The frame is rendered at 1× and drawn three times with 1 px offsets so the lines are 2–3 px.

Brief: "Show my three.js scene as a wireframe: EdgesGeometry with a 15° threshold, cyan lines, faces at 9 % opacity, a cyan floor grid, near-black background."

02 NEON

What: magenta tube-light outline with a soft glow.

How: the same edge lines in `#ff2fd6` over black faces (so hidden lines stay hidden), drawn onto a 2D canvas four times: twice blurred 22 px, once blurred 7 px, then sharp with ±1 px offsets.

Brief: "Make the wireframe look like neon: magenta lines on black faces, draw the frame blurred (22 px twice, 7 px once) under a sharp pass, dark floor grid."

03 CARTOON

What: flat cel-shaded colours with a black ink outline on a yellow card.

How: `MeshToonMaterial` with a 4-step gradient map (values 40 / 120 / 220 / 255). The outline is a second copy of every part in black `MeshBasicMaterial` with `side: BackSide`, scaled 1.04. A warm yellow ground disc catches the shadow. Background `#ffd166`.

Brief: "Cel-shade the scene: MeshToonMaterial with a 4-step gradient, back-face black shells scaled 1.04 for the ink line, flat yellow background with a shadow on the floor."

04 CLAY

What: soft matte "stop-motion" model on a pale table.

How: `MeshStandardMaterial`, roughness 0.95, every colour moved 32 % toward white, exposure 1.15, a pale ground disc with the soft shadow, a warm vignette in 2D. Background `#f4efe8`.

Brief: "Clay render: roughness 0.95, lighten the colours 32 %, soft shadow on a light ground disc, warm off-white background, gentle vignette."

05 PIXEL ART

What: chunky 8-bit sprite with faint scanlines.

How: render the scene at **128 × 228** and scale it up with `imageSmoothingEnabled = false`; `MeshLambertMaterial` gives hard shading steps; a dark line every pixel row at 18 %. Background `#2b2d42`.

Brief: "Pixel-art pass: render at 128×228 and upscale with image smoothing off, Lambert shading, faint scanlines, dark navy background."

06 ASCII

What: the rickshaw written in characters, terminal green.

How: render at 80 × 107, read the pixels, map brightness to `" .:-=+*#%@"`, draw bold Menlo characters in three greens on `#020b06`. A gamma of 0.6 keeps the mid-tones visible.

Brief: "ASCII pass: downsample to 80×107, map luminance to the 10-character ramp above, bold monospace glyphs in green on near-black, gamma 0.6."

07 BLUEPRINT

What: a side elevation on engineering blue with grids and dimension lines.

How: white edge lines over `#0b3d91`, faces in translucent lighter blue, a 48 px faint grid and a 240 px stronger grid, a white floor grid in 3D, and 2D dimension lines with the real numbers (2 625 mm, 1 785 mm) plus a title block.

Brief: "Blueprint look: side view, white edges, translucent blue faces, two overlaid grids, dimension lines with measurements and a small title block on #0b3d91."

08 VOXEL

What: the rickshaw as a few thousand small cubes.

How: one ray per (height, depth) row through the closed parts; the gaps between each part's entry and exit points are filled with 0.11 cubes in that part's colour (thin parts like pillars get one cube per row), as a single `InstancedMesh` with a tiny per-cube brightness jitter. Shadow on the floor.

Brief: "Voxelise my scene: cast a ray along X per grid row, fill between each mesh's entry and exit hits with 0.11 cubes coloured per mesh, one InstancedMesh, soft shadow."

09 PAPER CUT

What: flat paper layers with a drop shadow and paper grain.

How: `MeshBasicMaterial` colours lightened 18 %, black back-face outline, a blurred dark copy of the frame drawn 24 px right and 32 px down at 26 % as the shadow, a hashed noise pattern at 12 % for the grain. Paper `#f3eee4`.

Brief: "Paper-cut pass: flat lightened colours, thin black outline, a blurred dark copy offset 24/32 px at 26 % as a drop shadow, subtle paper grain."

10 GLITCH

What: the cartoon frame tearing with colour fringes and scanlines, with occasional bursts.

How: five horizontal slices per frame copied sideways by up to ±35 px (positions from a hash of the frame number, so it is repeatable), red and cyan copies at ±5 px in `screen` mode at 20 %, 2 px scanlines; about one frame in five is a burst with ten slices, ±80 px and stronger fringes.

Brief: "Glitch pass: 5 hashed horizontal slice shifts per frame, red/cyan channel copies at ±5 px in screen mode, 2 px scanlines, a stronger burst on one frame in five."

THE RULES THAT KEPT IT CLEAN

- **One scene, one camera, ten materials.** Never ten models.
- **Everything on the beat.** 112 BPM: the hook is 4 beats, every look is 4 beats, each cut and click lands on a beat.
- **Frame by frame.** Rendered with HyperFrames (`npx hyperframes render --resolution portrait-4k`): everything is a function of time, no randomness, no timers, so every render is identical.
- **Sound under, not over.** A quiet ACE-Step bed (local, royalty-free) and one soft click on every cut. No whooshes, no bass drops.

TOOLS (ALL FREE)

Claude Code · three.js r160 · a 2D canvas for the passes · HyperFrames for the render · ACE-Step 1.5 for the music

Questions? DM @thenispace. — NIS · AI · Tech · Productivity